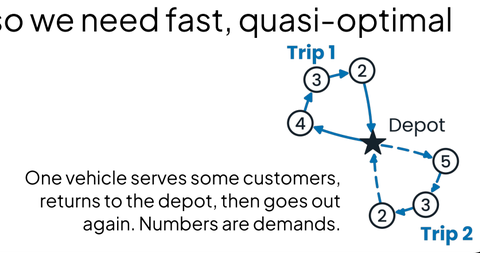
Group, Solve, Merge!

A decomposition heuristic for makespan-minimising multi-trip vehicle routing

Polikarpus Arya Pradhanika, Inamoto Tsutomu, Yoshinobu Higami, Ehime University

1 Motivation

- Given a depot, customers who need goods, and a limited fleet of vehicles, we plan how to deliver.
- With a limited fleet, one vehicle often makes several trips, and the day ends when the last vehicle is back.
- Exact solvers are slow, so we need fast, quasi-optimal solutions.



One vehicle serves some customers, returns to the depot, then goes out again. Numbers are demands.

2 Problem

- A depot and N customers, each at a known location with a known demand.
- K vehicles, each carrying at most Q units, may run several trips from the depot.
- Travel distance is the straight-line (Euclidean) distance between two points.
- Goal: minimise the makespan, the time the last vehicle returns.

Notation

N	number of customers
i, j	customer indices
(x_i, y_i)	location of customer i
q_i	demand of customer i
$d_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$	
K	number of vehicles
Q	vehicle's capacity
$T(r)$	duration of trip r

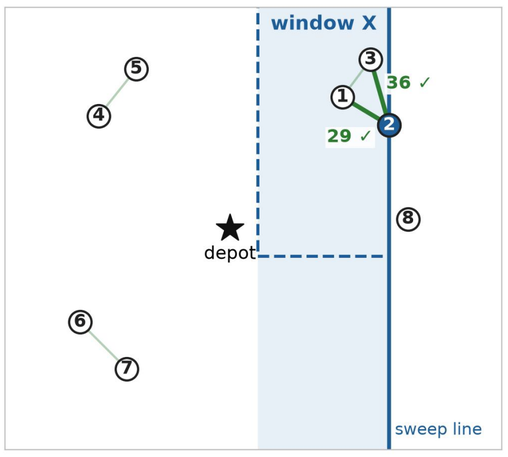
3 Method

Notation

E	number of edges	LB	lower bound	X	maximum group size	$E \le \frac{N(N-1)}{2}$	$\mathrm{UB} = \sum_{\forall r \in \mathcal{R}} T(r)$
G	number of groups after merging (one trip each)	UB	upper bound	T^*	makespan of final solution	$\mathcal{R}$ set of trips, $\lvert\mathcal{R}\rvert = G$	$\mathrm{LB} = \max\{\, T(r) \mid \forall r \in \mathcal{R} \,\}$

PRUNE

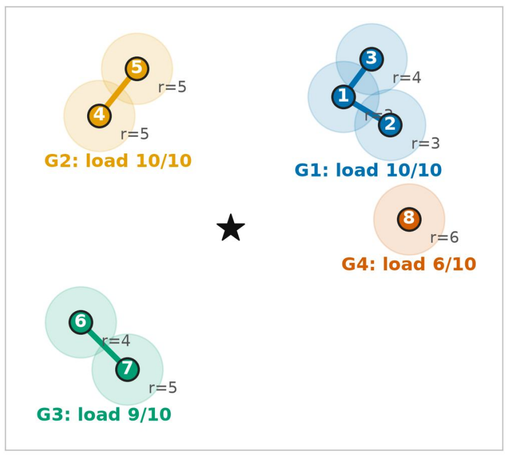
Line sweep keeps only short candidate edges



$O(N \log N + E)$

GROUP

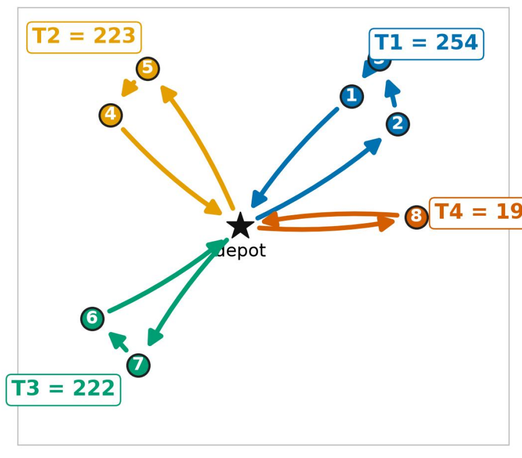
Kruskal merges customers while demand ≤ Q and size ≤ X



$O(E \log E)$

SOLVE

Held-Karp or Lin-Kernighan (LK) gives one tour per group

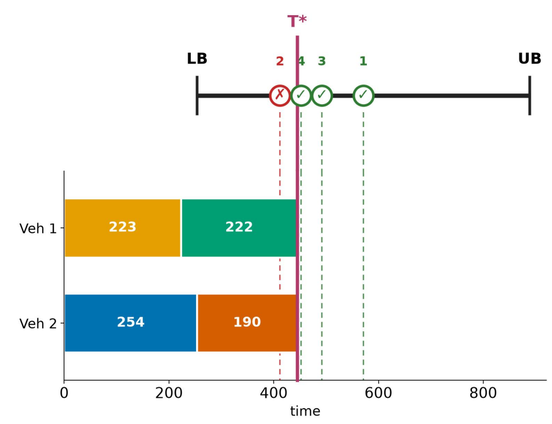


$O(GX^2 2^X)$ Held-Karp

$O(GX^{2.2})$ Lin-Kernighan

MERGE

Binary search on the makespan, pack trips onto vehicles



$O(G \log G + G \log(UB - LB))$

4 Setup

Instances (random)

- depot at (0,0)
- $-100 \le x_i,\, y_i \le 100, \quad x_i,\, y_i \in \mathbb{Z} \quad \forall i \in \{1, \ldots, N\}$
- $1 \le q_i \le 10, \quad q_i \in \mathbb{Z} \quad \forall i \in \{1, \ldots, N\}$
- one instance per cell, one run per method, sequentially

Two grids, 8×8 cells each

A grid varies two parameters over 8 values each. A cell is one pair of values, i.e. one random instance.

- N--K with $Q = 320$
- N--Q with $K = 20$

Metrics

- Makespan T^* (lower is better)
- Wall-clock time (s)

Methods

- Held-Karp ($X_{\max} = 10$)
- LK ($X_{\max} = 40$)
- PyVRP (ILS), stops after 20,000 iterations without improvement

Hardware and Libraries

Intel Core Ultra 7 155H (22 logical CPUs), 15 GiB RAM, Ubuntu 24.04 (WSL2). Python 3.13.13, NumPy 2.5.4, PyVRP 0.14.1, lk-heuristic 0.0.4.

5 Results

Makespan gap vs PyVRP (%)

K \ N	10	20	50	100	200	400	700	1k
100	+0	-46	-69	-59	-52	-69	-27	-56
50	+0	-46	-69	-59	-53	-45	-58	-60
20	+0	-46	-69	-73	-51	-38	-65	-36
10	+0	-46	-68	-59	-8	-24	-44	-15
5	+0	-46	-59	-27	+11	+12	-23	+4
3	+0	-46	-46	-13	+5	-4	-9	+8
2	+0	-26	-16	+14	-7	+24	-12	+19
1	+0	+1	+23	+23	+28	+24	+32	+30
1280	+0	-46	-68	-73	-85	-70	-67	-55
640	+0	-46	-68	-73	-75	-39	-44	-57
320	+0	-46	-69	-73	-51	-38	-65	-36
160	+0	-46	-53	-15	-33	-36	-30	-61
80	+0	-16	-42	+16	-5	-57	-62	-72
40	+0	+0	+15	+6	-49	-83	-85	-86
20	+0	-12	-13	-39	-66	-76	-67	-70
10	+13	+1	-71	-72	-72	-59	-55	-41

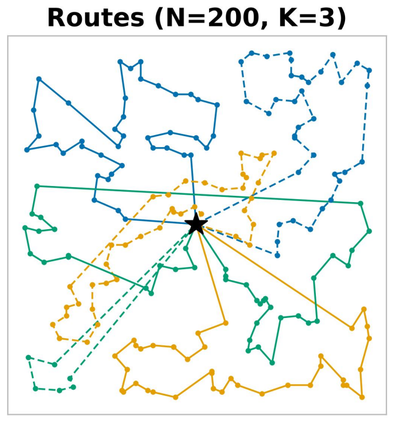
Q (capacity) / N (customers)

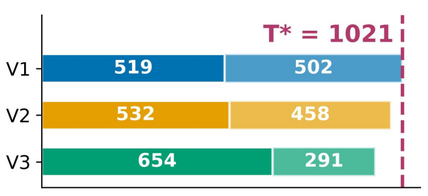
Speedup vs PyVRP (×)

K \ N	10	20	50	100	200	400	700	1k
100	63	140	133	73	41	29	17	18
50	94	146	116	67	48	22	17	54
20	84	103	103	39	74	26	43	45
10	50	136	155	93	23	16	53	98
5	111	197	139	62	37	48	42	117
3	108	145	120	111	91	67	127	111
2	90	132	135	66	58	72	44	78
1	93	131	101	136	129	47	195	77
1280	92	126	109	73	49	28	21	26
640	59	124	116	58	46	17	20	47
320	84	103	103	39	74	26	43	45
160	69	131	207	73	70	115	53	47
80	105	289	312	133	85	157	179	187
40	60	636	2.1k	1.0k	510	1.7k	631	575
20	590	2.5k	2.3k	1.5k	1.7k	973	864	1.3k
10	1.2k	1.8k	3.3k	1.8k	1.6k	1.3k	584	374

N (customers)

Routes (N=200, K=3)





How to Read

*Each cell is one instance (no averaging). Left: makespan gap = (ours – PyVRP) / PyVRP, using the better of Held-Karp and LK; **blue means we win**, **red means PyVRP wins.** Right: speedup = PyVRP time / our time; darker is faster.*

! Key Findings

Each cell is one random instance (no averaging). In 85 of the 128 cells, the better of Held-Karp and LK has a lower makespan than PyVRP (ILS), and it is at least 15× faster in all of them (median 103×). PyVRP is better mainly with 1–5 vehicles, where its objective (total distance) is closest to the makespan.

6 Limitation

- Two groups may fit in one trip $\sum q_i \le Q$ and finish sooner than two separate trips. Our method cannot merge trips this way.
- Exact methods (e.g. dynamic programming over subsets) only scale for small N *(≤ 20)*, so for larger N the gap to the optimum is unknown.

7 Future Works

- Merge trips into one when the capacity allows and the makespan improves.
- Compare with a baseline that minimises makespan, since PyVRP minimises total distance.

8 References

[1]. A. Laaksonen, Competitive Programmer's Handbook, 2018. [Online]. Available: https://cses.fi/book/book.pdf

Code and Data

github.com/mie-intel/ehime-works